Personal Trading Intelligence Platform

System Design Document v1.0

Author	Jaiminkumar Patel
Date	June 2026
Status	Design Phase — Pre-Implementation
Current Step	Step 1 Complete (Architecture) — Next: Step 2 (Data Model)
Hardware Target	MacBook Pro M5 Pro 48GB + VPS for always-on services
Primary Broker	Webull (via OpenAPI) — Robinhood later via SnapTrade

CONTEXT FOR CONTINUATION: This document captures the complete system design for a personal trading intelligence platform. If you are an AI assistant picking up this project, read this entire document first. The design is complete through Step 1 (High-Level Architecture + Component Map). The next step is Step 2: Data Model — defining the Postgres schema, prediction tracking tables, and the self-learning data structures. All design decisions below have been reviewed and approved.

Table of Contents

---

1. Project Vision and Goals

Build a personal trading intelligence platform that evolves from a portfolio tracker into an autonomous, self-improving options trading algorithm personalized to the user's risk tolerance, budget, and execution behavior.

Core Requirements

- **Portfolio Tracking** — live positions from Webull, real-time P&L;, sell signals for max profit
- **Options Discovery** — weekly scan of 3000+ optionable stocks, filter market cap > $50M, no penny stocks (price > $5)
- **Strategy Selection** — system autonomously decides best strategy (covered call, credit spread, iron condor, straddle, etc.) based on market conditions
- **Specific Contracts** — recommends exact strike price, entry price, stop loss, sell target
- **Self-Validation** — tracks every prediction, measures win/loss ratio, Sharpe ratio, learns from results
- **Autonomous Evolution** — eventually executes trades with user approval, personalized to individual trading profile

Design Principles

- **Minimize cloud LLM token spend** — use local LLM for heavy analysis, cloud only for conversation
- **MCP server architecture** — queryable from Claude, Claude Code, or any MCP-compatible client
- **Security first** — broker credentials encrypted at rest, zero secrets on server
- **Build own RAG** — ingest SEC filings, earnings transcripts, news, analyst reports for fundamental context
- **Subscription APIs OK** — willing to pay for quality market data (Polygon, Unusual Whales, etc.)

User Profile

- **Primary broker:** Webull (heavy usage)
- **Secondary broker:** Robinhood (occasional)
- **Options focus:** All strategies — system should recommend which strategy fits best
- **Ticker universe:** System discovers tickers — not limited to a watchlist
- **Filters:** Market cap > $50M, no penny stocks, sufficient options liquidity

2. Phase Roadmap

Phase	Name	Description	Key Deliverables
1	SEE	Portfolio tracking + sell signals	Webull integration, live positions, P&L tracking, exit signals
2	THINK	Discovery + strategy + recommendations	Scanner, strategy engine, specific contract recommendations, RAG
3	LEARN	Self-improving algorithm	Prediction tracking, pattern detection, personalized trading profile
4	ACT	Autonomous execution with approval	Trade execution via Webull API, user approval workflow, alerts

Each phase builds on the previous. Phase 1 infrastructure (broker connector, market data, database) is reused in all subsequent phases. The system is designed so every component can be developed and tested independently.

3. High-Level Architecture

The system follows a layered architecture with the MCP Server as the central orchestration point. All interactions flow through the MCP server, which routes requests to the appropriate internal service.

Architecture Layers

Layer	Purpose	Components
Client Layer	User interaction	Claude Desktop, Claude Code, Custom Dashboard (future)
MCP Server	Orchestration, tool routing	Go or Python MCP server, 9 tool endpoints
Broker Layer	Personal financial data	Webull OpenAPI (OAuth2), SnapTrade for Robinhood (later)
Market Data Layer	Market prices, options, technicals	Polygon.io (paid), Yahoo Finance (free backup)
Research Layer (RAG)	Fundamental context	SEC EDGAR, earnings transcripts, news, ChromaDB, local embeddings
Intelligence Engine	Analysis and reasoning	Local LLM (Qwen 14B), Technical analysis, Strategy engine
Storage Layer	Persistence	Postgres (encrypted), ChromaDB for embeddings

MCP Tool Endpoints

Tool	Phase	Description
`get_positions`	1	Live portfolio positions from Webull
`get_balances`	1	Account cash, buying power, margin
`get_orders`	1	Open and filled order history
`get_sell_signals`	1	When to exit current positions for max profit
`get_portfolio_pnl`	1	Overall portfolio performance and P&L
`discover_options`	2	Weekly scan — top 10-20 options opportunities
`analyze_ticker`	2	Deep dive on one ticker (technicals + fundamentals)
`get_recommendation`	2	Specific contract: strategy, strike, entry, SL, target
`get_options_chain`	2	Full options chain with greeks for a ticker
`get_market_regime`	2	Current market conditions (bull/bear/sideways, VIX)
`explain_strategy`	2	Natural language explanation of a strategy recommendation
`get_prediction_log`	3	Historical predictions and their outcomes
`get_accuracy_report`	3	Win/loss ratio, Sharpe, per strategy performance
`get_user_profile`	3	Risk tolerance, budget, execution behavior analysis
`backtest_strategy`	3	Backtest a strategy on historical data

`get_learnings`	3	What the system has learned about what works
`execute_trade`	4	Place a trade on Webull (with user approval)
`adjust_algorithm`	4	Manually adjust strategy weights or parameters

4. Component Map — 12 Components

The system consists of 12 independent components. Each can be developed, tested, and deployed separately. The primary language is Python. The MCP server can be Go or Python.

#	Component	Purpose	Phase	Depends On
1	MCP Server	Entry point, tool routing	1	All components
2	Broker Connector	Webull API (positions, orders, P&L)	1	Webull OAuth2
3	Market Data Service	Quotes, options chains, historical	1	Polygon.io API
4	Options Flow Service	Unusual activity, institutional flow	2	Unusual Whales API
5	Scanner / Discovery	Weekly scan, filter, rank candidates	2	C3, C4, C6
6	Technical Analysis	Trend, S/R, RSI, MACD, patterns	1	C3 (price data)
7	Strategy Engine	Decide strategy + specific contracts	2	C6, C8, C9
8	RAG Pipeline	Ingest docs, embed, query context	2	ChromaDB, Ollama
9	Local LLM Service	Ollama + Qwen 14B for reasoning	1	Ollama
10	Prediction Tracker	Log every recommendation + outcome	2	C12 (Postgres)
11	Learning Engine	Pattern detection, weight adjustment	3	C10, C12
12	Postgres Database	All persistence, encrypted creds	1	None

Component Interaction Flow

User asks Claude a question (e.g., 'What are the best options plays this week?')

```
Claude calls MCP Server
MCP Server calls C5: Scanner / Discovery Engine
C5 calls C3: Market Data (price, volume, IV for 3000+ stocks)
C5 calls C4: Options Flow (unusual activity)
C5 calls C6: Technical Analysis (trend, S/R, momentum)
C5 filters: market cap > $50M, price > $5, options liquidity
C5 returns top 10-20 candidates
MCP Server calls C7: Strategy Engine (for each candidate)
C7 calls C6: Technicals (detailed analysis)
C7 calls C8: RAG (earnings, news, SEC filings)
C7 calls C9: Local LLM (reasoning about best strategy)
C7 returns: strategy, strike, entry, stop loss, target
MCP Server calls C10: Prediction Tracker (log recommendation)
MCP Server returns results to Claude
Claude presents recommendations to you in natural language
```

5. Component Details

Component 1 — MCP Server

The central orchestration layer. All tools are exposed via MCP protocol. Routes requests to the appropriate internal component. Handles authentication, rate limiting, and error handling.

- **Language:** Go or Python (FastMCP)
- **Protocol:** MCP (Model Context Protocol)
- **Deployment:** VPS for always-on access, or local MacBook for dev
- **Tools:** 18 tool endpoints across all 4 phases (see Section 3)

Component 2 — Broker Connector

Integrates with Webull's OpenAPI for personal financial data. Read-only in Phase 1-3, trade execution in Phase 4. Robinhood added later via SnapTrade aggregator.

- **Webull OpenAPI:** OAuth2 authentication, Python/Java SDK available
- **Endpoints used:** positions, balances, orders, account info, trade execution (Phase 4)
- **Data:** positions with cost basis, unrealized P&L;, order history, buying power
- **Security:** OAuth2 tokens encrypted at rest in Postgres, refresh handled automatically
- **Rate limits:** respect Webull's API rate limits, cache where possible

Component 3 — Market Data Service

Provides real-time and historical market data for all analysis components. Primary source is Polygon.io ($29/month) with Yahoo Finance as free fallback.

- **Real-time:** quotes, last price, bid/ask, volume
- **Options:** full options chains with greeks (delta, gamma, theta, vega), IV, OI
- **Historical:** OHLCV bars (1min to monthly), adjusted for splits/dividends
- **Caching:** frequently accessed data cached locally to reduce API calls
- **Polygon protocols:** HTTP for standard, MQTT for real-time streaming

Component 4 — Options Flow Service

Detects unusual options activity that signals institutional interest. Key input for the discovery engine to find high-probability setups.

- **Source:** Unusual Whales or FlowAlgo ($40-70/month)
- **Signals:** volume spikes, large block trades, dark pool activity, put/call ratio shifts
- **Integration:** API polling on schedule, results cached and scored

Component 5 — Scanner / Discovery Engine

The weekly (or daily) scan that finds the best options opportunities from the entire optionable stock universe. This is the funnel that narrows 3000+ stocks to 10-20 candidates.

- **Universe:** all optionable US stocks (~3000+)
- **Filters:** market cap > $50M, price > $5, minimum options OI and volume
- **Scoring:** combines technical signals, IV rank, unusual flow, earnings proximity
- **Output:** ranked list of 10-20 candidates with reasons
- **Schedule:** weekly full scan (weekend), daily quick scan (pre-market)
- **Runs locally:** pure computation + local LLM, no cloud API needed

Component 6 — Technical Analysis Engine

Pure computational analysis of price action. No LLM needed — all math and algorithms.

- **Indicators:** MA (20, 50, 200), EMA, MACD, RSI, Bollinger Bands, ATR, VWAP
- **Patterns:** support/resistance levels, trend channels, breakout/breakdown detection
- **Volume:** volume profile, relative volume, accumulation/distribution
- **Library:** ta-lib or pandas-ta in Python
- **Performance:** runs in milliseconds per ticker, no bottleneck

Component 7 — Strategy Engine

The decision-making core. Takes a ticker + market conditions and determines the optimal options strategy with specific contract details.

- **Input:** ticker, technicals (from C6), fundamentals (from C8/RAG), market regime, IV data
- **Decision tree:** IV rank high (>60) + sideways = sell premium (iron condor, credit spread). IV rank low + trending = buy directional (long calls/puts, debit spreads). Earnings imminent = straddle/strangle or avoid.
- **Output:** strategy name, specific strikes, expiration, entry price, stop loss, profit target, risk/reward ratio
- **Uses local LLM:** for reasoning about edge cases and explaining the rationale
- **Configurable:** risk tolerance, max position size, max loss per trade (from user profile)

Component 8 — RAG Pipeline

Ingests financial documents into a vector database for fundamental context. When analyzing a ticker, the system queries relevant documents — not just price data but WHY the stock is moving.

- **Data sources:** SEC EDGAR (10-K, 10-Q, 8-K filings), earnings call transcripts, financial news, analyst reports
- **Vector DB:** ChromaDB (local, free)
- **Embedding model:** nomic-embed-text via Ollama (local, free)
- **Chunking:** document-aware splitting (sections, paragraphs, tables)
- **Query:** when analyzing a ticker, retrieve top-k relevant chunks for context
- **Update frequency:** SEC filings on publish, earnings transcripts quarterly, news daily

Component 9 — Local LLM Service

Runs locally on your MacBook via Ollama. Handles all analysis reasoning to avoid cloud API costs. Cloud LLM (Claude) is only used for the conversation layer with you.

- **Model:** Qwen 14B or Llama 3.1 8B (fits comfortably in 48GB with other services)
- **Use cases:** strategy reasoning, ticker analysis summaries, natural language explanations
- **Not used for:** scanning (pure computation), technical analysis (math), options pricing (formulas)
- **Coding assistant:** Qwen 2.5 Coder 14B for development tasks

Component 10 — Prediction Tracker

THE MOST CRITICAL COMPONENT. Logs every single recommendation with full context. Tracks outcomes. This is the data foundation for the learning engine.

- **Logs on every recommendation:** ticker, strategy, strikes, entry, stop loss, target, IV rank, IV percentile, days to expiry, delta, market regime, VIX level, sector, earnings proximity, unusual flow signal, technical setup type, timestamp
- **Tracks outcomes:** final P&L;, hit target or stop loss, days held, actual vs predicted move, early exit reason (if any)
- **Metrics:** win rate, avg return, max drawdown, Sharpe ratio, profit factor
- **Granularity:** per strategy, per market regime, per IV environment, per sector, per ticker

Component 11 — Learning Engine (Phase 3)

Analyzes prediction tracker data to find patterns and adjust the algorithm's strategy weights.

- **Pattern detection:** 'iron condors on high IV rank (>60) in sideways markets win 72%'
- **Weight adjustment:** strategies that consistently win get higher recommendation scores
- **User profile learning:** tracks which strategies YOU execute well vs exit early, adjusts recommendations accordingly
- **Market regime awareness:** learns that different strategies work in different environments
- **Minimum data:** needs 50+ predictions before patterns are reliable, 200+ for personalization
- **Feedback loop:** every new outcome updates the model, no manual retraining needed

Component 12 — Postgres Database

Central persistence layer. All data encrypted at rest. Stores everything the system needs to remember across sessions.

- **Encrypted credentials:** Webull OAuth tokens, API keys
- **Prediction log:** every recommendation + context + outcome
- **User profile:** risk tolerance, budget, position sizing rules, execution behavior
- **Strategy performance:** historical win rates per strategy per condition
- **Market regime history:** VIX levels, trend direction, breadth at time of each prediction
- **RAG metadata:** document tracking (what's been ingested, when)

6. Data Sources and Subscriptions

Service	Cost/month	What You Get	Phase	Required?
Webull OpenAPI	Free	Positions, orders, balances, trade exec	1	Yes
Polygon.io	$29	Real-time quotes, options chains, greeks, historical, unusual activity	1	Yes
Unusual Whales or FlowAlgo	$40-70	Options flow, institutional activity, dark pool data	2	Recommended
Seeking Alpha or TipRanks	$20-30	Analyst ratings, earnings estimates, research reports for RAG	2	Optional
SEC EDGAR	Free	10-K, 10-Q, 8-K filings	2	Yes
Yahoo Finance	Free	Backup quotes, basic options chains	1	Free backup
SnapTrade	Free tier	Robinhood integration (later)	Future	Optional

Total estimated monthly cost: ~$90-130/month for institutional-grade data. Compare that to one bad options trade.

7. Token Usage Strategy

90% of computation runs locally. Cloud LLM is only the conversation interface.

Task	Where It Runs	Cost
Your conversation with Claude	Cloud (Claude API)	~$5-10/month
Ticker scanning (3000+ stocks)	Local (pure computation)	Free
Technical analysis	Local (Python, ta-lib)	Free
Options chain math / Greeks	Local (Python)	Free
Strategy reasoning	Local LLM (Qwen 14B)	Free
RAG embedding	Local (nomic-embed-text)	Free
RAG query + context retrieval	Local (ChromaDB)	Free
Natural language explanations	Local LLM (Qwen 14B)	Free
Prediction tracking	Local (Postgres)	Free
Learning engine	Local (Python + stats)	Free

8. Self-Learning Loop Design (Most Critical)

This is what separates this system from a simple screener. Every recommendation is tracked, every outcome is measured, and the algorithm adjusts based on what actually works.

Learning Timeline

Timeframe	Data Points	What Happens
Week 1-4	10-20 predictions	System logs everything, no learning yet. Just collecting baseline data.
Month 2-3	50+ predictions	Patterns start emerging. 'Iron condors on high IV win 72%'. Basic strategy weighting begins.
Month 3-6	100-200 predictions	Strategy weights refined. Market regime awareness kicks in. User execution patterns detected.
Month 6+	200+ predictions	Fully personalized algorithm. Knows YOUR risk tolerance, execution style. Recommendations tailored to you.

Learning Dimensions

The system tracks performance across multiple dimensions to find what works:

- **By strategy:** iron condor win rate vs credit spread vs straddle
- **By IV environment:** high IV (>60) vs medium (30-60) vs low (<30)
- **By market regime:** trending up vs trending down vs sideways
- **By VIX level:** low vol (<15) vs normal (15-25) vs high vol (>25)
- **By sector:** tech options behave differently than energy or healthcare
- **By earnings proximity:** pre-earnings, post-earnings, no earnings nearby
- **By days to expiry:** weeklies vs monthlies vs LEAPs
- **By your execution:** did you follow the plan or exit early? Why?

Example Learning Cycle

```
Week 1: System recommends NVDA iron condor
Logs: ticker=NVDA, strategy=iron_condor, IV_rank=72,
regime=sideways, VIX=18, sector=tech, DTE=14

Week 2: Position hits profit target at 50% max profit
Updates: iron_condor + high_IV + sideways = +1 score
tech_sector + IV_rank>60 = +1 score

Week 3: System recommends AMD credit spread
Logs: ticker=AMD, strategy=credit_spread, IV_rank=55,
regime=trending_up, pre_earnings=true

Week 4: Position hits stop loss after earnings surprise
Updates: credit_spread + pre_earnings = -1 score
LESSON: avoid credit spreads 7 days before earnings
```

9. Technology Stack

Category	Technology	Why
Language	Python (primary)	Best ecosystem for finance + ML + APIs
MCP Server	Python (FastMCP) or Go	Go for performance, Python for speed of development
Database	PostgreSQL	Reliable, encrypted at rest, JSON support
Vector DB	ChromaDB	Local, free, Python-native, good enough for RAG
Local LLM	Ollama + Qwen 14B	Fits in 48GB, strong reasoning, multilingual
Embeddings	nomic-embed-text (Ollama)	Local, free, good quality for finance docs
Technical Analysis	ta-lib or pandas-ta	Industry standard indicators
Options Pricing	py_vollib or mibian	Black-Scholes, greeks calculation
Data Processing	pandas, numpy	Standard data manipulation
Market Data	Polygon.io Python SDK	Official SDK, well documented
Broker	Webull OpenAPI SDK	Official Python SDK
Deployment	Docker + VPS	Always-on for alerts and scheduled scans
Dev Machine	MacBook M5 Pro 48GB	Runs local LLM + full dev stack

10. Open Decisions for Step 2 (Data Model)

The following decisions need to be made in Step 2 when defining the database schema and data structures:

#	Decision	Options	Notes
1	MCP Server language	Go vs Python (FastMCP)	Go = performance. Python = faster dev, same ecosystem
2	Prediction log granularity	Per-leg vs per-strategy	Iron condor: log as 1 trade or 4 individual legs?
3	RAG update frequency	Real-time vs batch	News = real-time. SEC filings = on publish. Earnings = quarterly.
4	User profile schema	Explicit config vs learned	Start explicit (risk=medium), learn preferences over time
5	Backtesting engine	Build vs use existing (backtrader, zipline)	Build custom for options-specific backtesting
6	Alert delivery	Slack, email, SMS, push notification	Which channels for real-time sell signals?
7	VPS provider	DigitalOcean, Hetzner, Linode, AWS Lightsail	Need always-on for scans and alerts. $10-20/month.

CONTINUATION INSTRUCTIONS FOR AI ASSISTANT:

This document represents the completed output of **Step 1: High-Level Architecture**.

Step 2: Data Model is the next step. It should cover:
- Postgres schema for all tables (predictions, outcomes, user profile, strategy weights, market regimes)
- Prediction log structure — what gets logged on every recommendation
- Outcome tracking structure — what gets recorded when a trade closes
- Learning engine data structures — how weights are stored and updated
- RAG document schema — how ingested documents are tracked
- User profile schema — risk tolerance, budget, execution behavior

The user (Jaimin) is a Staff Engineer at PayPal with deep Java/reactive programming experience. He prefers Python for this project. He uses Webull as his primary broker. He wants the system to discover tickers autonomously (market cap > $50M, no penny stocks) and recommend specific options strategies with exact contracts. The self-learning loop (prediction tracking + outcome validation) is the most critical feature. Minimize cloud LLM costs — use local Qwen 14B for analysis, cloud Claude only for conversation.